



SAÉ 1.05 – TRAITER DES DONNÉES

CRÉATION D'UN OUTIL DE « REPORTING » QUI PERMET DE LOCALISER LES GROS
FICHIERS DU DISQUE

KIENTZ ALEXANDRE, ERB LOUIS, LEGOLL BENJAMIN

SOMMAIRE

1. Présentation du projet
2. Script Powershell
3. 1^{er} Script Python
4. 2^e Script Python
5. 3^e Script Python
6. Conclusion

PRÉSENTATION DU PROJET

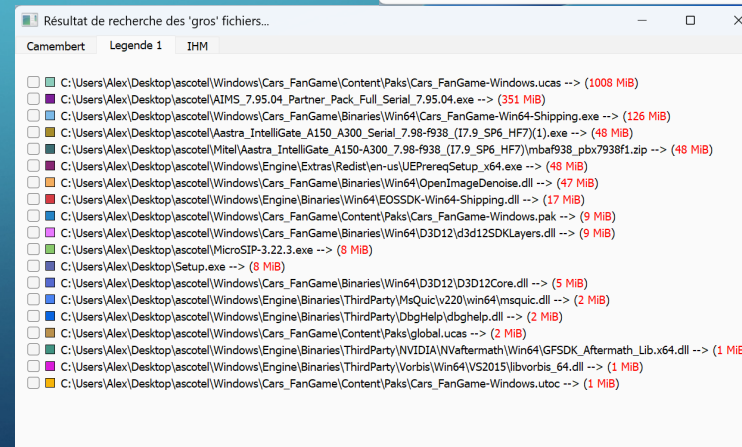
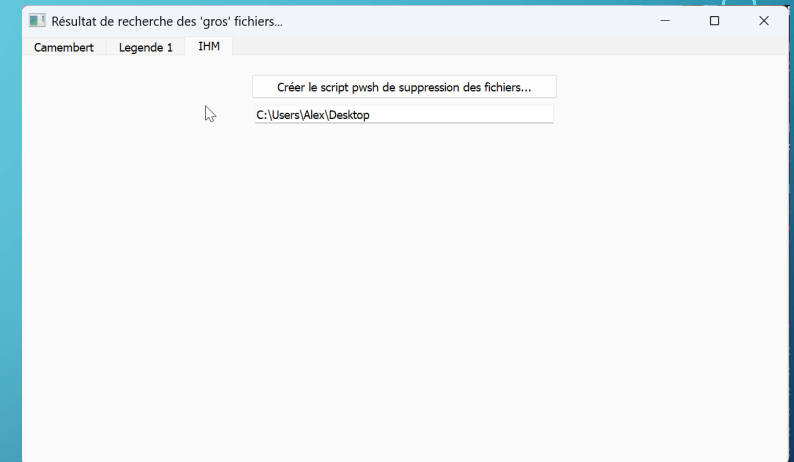
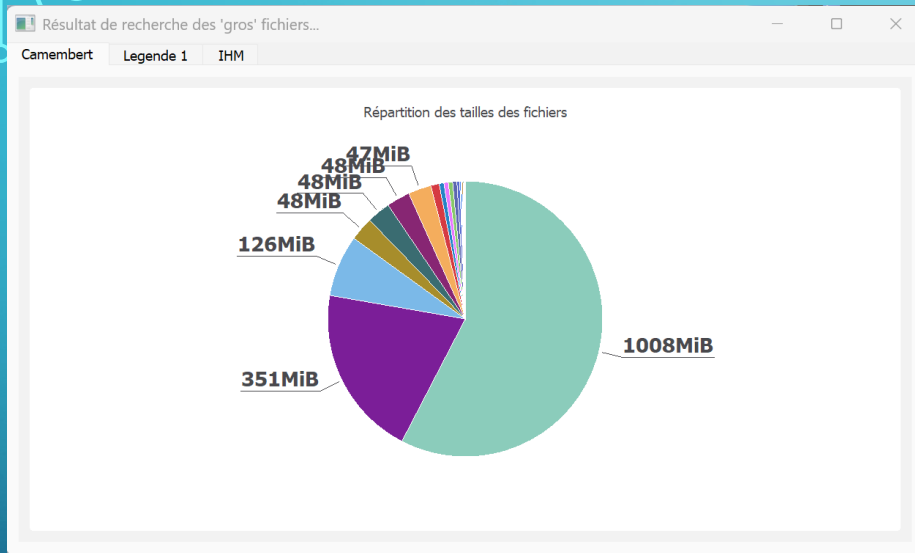
Objectifs :

- Résoudre le manque d'espace disque peut bloquer le fonctionnement normal d'un PC
- Les fichiers volumineux et inutilisés occupent une grande partie du stockage
- Le programme analyse un répertoire et permet de repérer puis supprimer les 100 plus gros fichiers au-dessus d'un seuil défini

Contraintes :

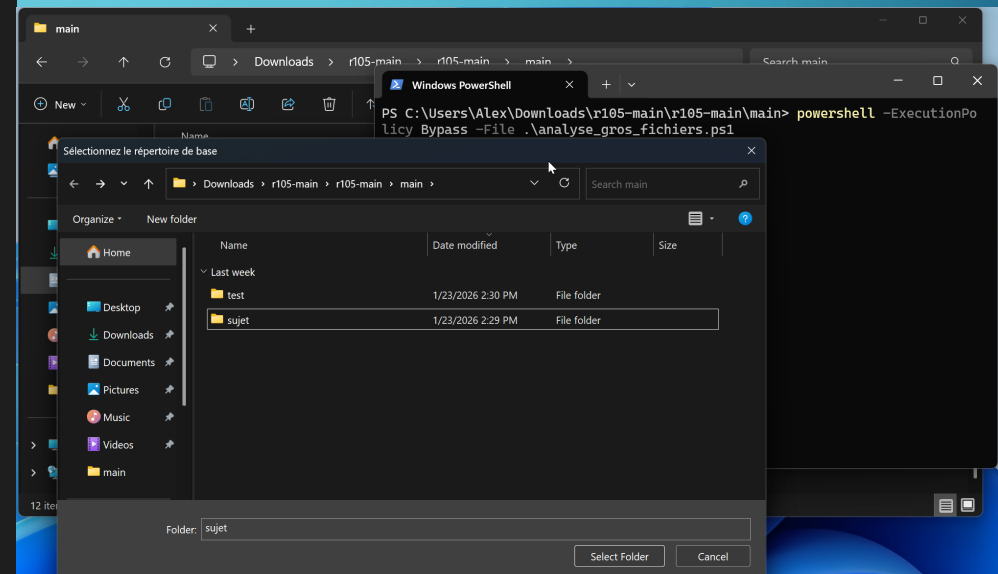
- L'outil est réservé à des utilisateurs avertis car la suppression de fichiers systèmes peut être dangereuse
- Utilisation du PowerShell
- Le programme doit analyser, trier, afficher graphiquement les 100 plus gros fichiers et générer un script PowerShell de suppression

PRÉSENTATION DU PROJET



SCRIPT POWERSHELL

```
7 # Paramètres de base
8 $MIN_SIZE_MIB = 1
9 $MAX_FILES = 100
10
11 # Se placer dans le dossier du script
12 $scriptDir = Split-Path -Parent $MyInvocation.MyCommand.Path
13 Set-Location $scriptDir
14
15 $jsonOut = Join-Path $scriptDir "report.json"
16
17 # 1) Sélection du répertoire
18 $rep_base = & py (Join-Path $scriptDir "script_1_selection_repertoire.py")
19
20 # le dossier doit exister
21 if (-not (Test-Path -LiteralPath $rep_base)) {
22     Write-Output "Repertoire invalide ou non selectionne: $rep_base"
23     exit 1
24 }
25
26 # 2) Scan + JSON
27 & py (Join-Path $scriptDir "script_2_scan_et_json.py") "$rep_base" $MIN_SIZE_MIB $MAX_FILES "$jsonOut"
28
29 # 3) IHM depuis JSON
30 & py (Join-Path $scriptDir "script_3_affichage_ihm.py") "$jsonOut" "$rep_base"
```



1^{ER} SCRIPT PYTHON

```
8
9  import sys
10 from pathlib import Path
11
12 from PyQt5.QtWidgets import QApplication, QFileDialog
13
14
15 def choisir_repertoire():
16     dossier = QFileDialog.getExistingDirectory(None, "Sélectionnez le répertoire de base", "")
17     if dossier:
18         return str(Path(dossier).resolve())
19     return ""
20
21
22 def main():
23     app = QApplication(sys.argv)
24     rep_base = choisir_repertoire()
25     print(rep_base)
26     return 0
27
28
29 if __name__ == "__main__":
30     raise SystemExit(main())
31
```

2^E SCRIPT PYTHON

```
10
11 import json
12 import sys
13 from pathlib import Path
14
15
16 def inventorie_fichiers(repertoire_de_base):
17     base = Path(repertoire_de_base).resolve()
18     liste = []
19     for p in base.rglob("*"):
20         if p.is_file():
21             liste.append([str(p.resolve()), p.stat().st_size])
22     return liste
23
24
25 def tri_decroissant_par_taille(liste_fichiers):
26     return sorted(liste_fichiers, key=lambda x: x[1], reverse=True)
27
28
29 def filtre_et_limite(liste_fichiers, taille_mini_mib, nb_maxi_fichiers):
30     seuil_octets = taille_mini_mib * 1048576
31     liste_filtre = [x for x in liste_fichiers if x[1] >= seuil_octets]
32     return liste_filtre[:nb_maxi_fichiers]
33
```

```
33
34
35 def ecrit_json(liste_fichiers, chemin_json):
36     with open(chemin_json, "w", encoding="utf-8") as f:
37         json.dump(liste_fichiers, f, ensure_ascii=False, indent=2)
38
39
40 def main():
41     liste = inventorie_fichiers(sys.argv[1])
42     liste = tri_decroissant_par_taille(liste)
43     liste = filtre_et_limite(liste, int(sys.argv[2]), int(sys.argv[3]))
44     ecrit_json(liste, sys.argv[4])
45     return 0
46
47
48 if __name__ == "__main__":
49     raise SystemExit(main())
50
```

3^E SCRIPT PYTHON

```
7
8 import json
9 import random
10 import sys
11 from pathlib import Path
12
13 from PyQt5.QtGui import QColor
14 from PyQt5.QtWidgets import QApplication
15
16 # Les 4 fichiers fournis par le sujet sont dans ./sujet/
17 SUJET_DIR = Path(__file__).resolve().parent / "sujet"
18 if str(SUJET_DIR) not in sys.path:
19     sys.path.insert(0, str(SUJET_DIR))
20
21 from Creation_Onglets import Onglets
22 from Creation_Camembert import Camembert
23 from Creation_Legendes import Legendes
24 from Creation_Boutons import Boutons
25
26 NB_LEGENDES_PAR_PAGE = 25
27
28
29 def lit_json(chemin_json):
30     path = Path(chemin_json).expanduser().resolve()
31     with path.open("r", encoding="utf-8") as f:
32         return json.load(f)
33
34
35 def genere_couleurs(nb):
36     return [QColor(random.randint(0, 255), random.randint(0, 255), random.randint(0, 255)) for _ in range(nb)]
```

```
39 def ecrit_script_suppression(liste_fichiers_a_supprimer, chemin_ps1):
40     lignes = [
41         'Write-Output "Script PowerShell pour supprimer des fichiers sans confirmation"',
42         'Write-Output "Attention : cette suppression est definitivement ..."',
43         '$reponse = Read-Host "Veuillez confirmer la suppression de tous ces fichiers : (OUI)"',
44         'if ($reponse -eq "OUI") {',
45         '    $confirmation = Read-Host "Etes-vous bien certain(e) ? (OUI)"',
46         '    if ($confirmation -eq "OUI") {'
47     ]
48     for path_str in liste_fichiers_a_supprimer:
49         lignes.append(f'        Remove-Item -Path "{path_str}" -Force')
50     lignes += [
51         '    } else {',
52         '        Write-Output "Operation annulee..."',
53         '    }',
54     ]
55     '}' else {',
56     '    Write-Output "Operation annulee..."',
57     '}'
58     open(chemin_ps1, "w", encoding="utf-8").write("\n".join(lignes) + "\n")
59
```


3^E SCRIPT PYTHON

```
def main():
    app = QApplication(sys.argv)
    liste_fichiers = lit_json(sys.argv[1])
    couleurs = genere_couleurs(len(liste_fichiers))

    fenetre = Onglets()

    # Onglet camembert
    fromage = Camembert(liste_fichiers, couleurs)
    layout_fromage = fromage.dessine_camembert()
    fenetre.add_onglet("Camembert", layout_fromage)

    # Onglets légendes (25 par page)
    pages_legendes = []
    nb_pages = (len(liste_fichiers) + NB_LEGENDES_PAR_PAGE - 1) // NB_LEGENDES_PAR_PAGE
    for page in range(nb_pages):
        start = page * NB_LEGENDES_PAR_PAGE
        obj = Legendes(liste_fichiers, couleurs, start, NB_LEGENDES_PAR_PAGE)
        pages_legendes.append(obj)
        layout_leg = obj.dessine_legendes()
        fenetre.add_onglet(f"Legende {page + 1}", layout_leg)

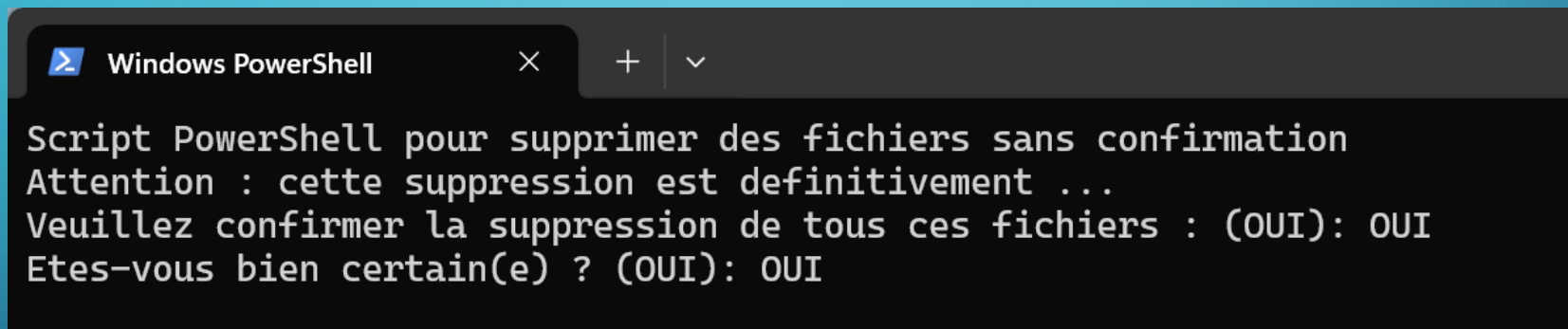
    # Onglet IHM (bouton)
    def on_click():
        fichiers_a_supprimer = []
        for page_idx, page_obj in enumerate(pages_legendes):
            etats = page_obj.recupere_etats_cases_a_cocher()
            start_idx = page_idx * NB_LEGENDES_PAR_PAGE
            for offset, checked in enumerate(etats):
                if checked:
                    fichiers_a_supprimer.append(liste_fichiers[start_idx + offset][0])
        ecrit_script_suppression(fichiers_a_supprimer, "delete_selected.ps1")

    ihm = Boutons(sys.argv[2], on_click)
    layout_ihm = ihm.dessine_boutons()
    fenetre.add_onglet("IHM", layout_ihm)

    fenetre.show()
    return app.exec_()

if __name__ == "__main__":
    raise SystemExit(main())
```

CONCLUSION



```
Windows PowerShell
Script PowerShell pour supprimer des fichiers sans confirmation
Attention : cette suppression est definitivement ...
Veuillez confirmer la suppression de tous ces fichiers : (OUI): OUI
Etes-vous bien certain(e) ? (OUI): OUI
```